Applied Mathematics and Computation 224 (2013) 36–45

Contents lists available at ScienceDirect

Applied Mathematics and Computation

journal homepage: www.elsevier.com/locate/amc

# The steepest descent algorithm without line search for p-Laplacian

Guangming Zhou [a,b,*], Chunsheng Feng [a]

[a] School of Mathematics and Computational Science, Xiangtan University, Xiangtan 411105, Hunan, PR China
[b] Hunan Key Laboratory for Computation and Simulation in Science and Engineering, Xiangtan University, Xiangtan 411105, Hunan, PR China

ARTICLE INFO

*Keywords:*
p-Laplacian
Continuation method
Finite element approximation
The steepest descent algorithm
Multigrid solver
Numerical experiment

ABSTRACT

In this paper, the steepest descent algorithm without line search is proposed for p-Laplacian. Its search direction is the weighted preconditioned steepest descent one, and step length is estimated by a formula except the first iteration. Continuation method is applied for solving the p-Laplacian with very large $p$. Lots of numerical experiments are carried out on these algorithms. All numerical results show the algorithm without line search can cut down some computational time. Fast convergence of these new algorithms is displayed by their step length figures. These figures show that if search direction is the steepest descent one, exact step lengths can be substituted properly with step lengths obtained by the formula.

© 2013 Elsevier Inc. All rights reserved.

## 1. Introduction

The form of the p-Laplacian with Dirichlet data is as follows

$$-div(|\nabla u|^{p-2}\nabla u) = f, \quad \text{in } \Omega, u = 0, \quad \text{on } \partial\Omega, \tag{1}$$

where $\Omega$ is a bounded open subset of $R^2$ with a Lipschitz boundary $\partial\Omega, 1 < p < +\infty, f \in L^2(\Omega)$, and $|\cdot|^2 = (\cdot,\cdot)_{R^2}$. It is well known that the Eq. (1) is Poisson equation when $p = 2$. The Eq. (1) is an important nonlinear equation. It comes from many physical problems, for instance, nonlinear diffusion and filtration (See [1]), power-law materials (see [2]), and quasi-Newtonian flows (see [3]). There are lots of literature on finite element approximation of the p-Laplacian. One can see the literatures [1–11] and the references therein. Of course, some results concerning sharp a priori and a posteriori error bounds of the equation owe to quasi-norm approach (see [12–14]).

The problem (1) is equivalent to the following minimization problem:

$$\min_{v\in V} J(v), \tag{2}$$

where

$$J(v) = \frac{1}{p}\int_\Omega |\nabla v|^p - \int_\Omega f v, \tag{3}$$

and $V$ is the Sobolev space $W_0^{1,p}(\Omega)$. The space is the closure of the space $C_0^\infty(\Omega)$ in the space $W^{1,p}(\Omega)$. Given a two-index $\alpha = (\alpha_1, \alpha_2) \in N^2$, we let $|\alpha| = \sum_{i=1}^2 \alpha_i$. When equipped with the norm

* Corresponding author.
*E-mail addresses:* zhougm@xtu.edu.cn, seabird_0605@163.com (G. Zhou).

0096-3003/$ - see front matter © 2013 Elsevier Inc. All rights reserved.
http://dx.doi.org/10.1016/j.amc.2013.07.096

$$\|v\|_{1,p,\Omega} = (\sum_{|\alpha|\leq 1} \int_\Omega |\partial^\alpha v|^p dx)^{\frac{1}{p}},$$

the space $W^{1,p}(\Omega)$ is a Banach space which, for any number $p$ satisfying $1 < p < +\infty$, consists of those functions $v \in L^p(\Omega)$ for which all partial derivatives $\partial^\alpha v$ with $|\alpha| \leqslant 1$ belong to the space $L^p(\Omega)$. One can refer many references, for example, [15], for the details of these symbols $W_0^{1,p}(\Omega), \partial^\alpha v$ etc.

A direct calculation yields that

$$J'(v)(\tilde{v}) = \int_\Omega |\nabla v|^{p-2} \nabla v \nabla \tilde{v} - \int_\Omega f\tilde{v}, \quad \forall\ \tilde{v} \in V. \tag{4}$$

The problem (2) has a unique solution which is also the solution of the system (1) (See [15]). The form of the Eq. (1) is simple. But how to solve it numerically is a very difficult problem when $p$ is very big ($p \gg 2$) and $p$ is close to $1^+$. The status is improved primely since the steepest descent algorithm [16] is proposed. The algorithm can effectively solve the equivalent problem of the p-Laplacian with large $p$, and has some good properties. Its basic thought will be given in next section. Based on the weighted preconditioned steepest descent direction, preconditioned hybrid conjugate gradient algorithm for the p-Laplacian is proposed [17], and it has higher efficiency than the algorithm [16] when $p$ is large. In order to solve the problem (2), we introduce the following finite element space like [15]. Let $T^h$ be a regular triangulation of $\Omega_h$, which is composed of disjoint open regular triangles $K_i$, that is, $\bar{\Omega}_h = \bigcup_{K_i \in T^h} \bar{K}_i$, where $h = \max_{K\in T^h} h_K$, and $h_K$ is the diameter of the element $K$ in $T^h$. $\bar{K}_i \bigcap \bar{K}_j$ is void, or only one common vertex, or a whole edge when $i \neq j$. In this paper, we shall only discuss the continuous piecewise linear element. Let $V^h$ be a finite dimensional subspace of $C^0(\bar{\Omega}_h)$, in which $\chi|_K \in \mathcal{P}_1$ if $\chi \in V^h$ and $\mathcal{P}_1$ is the linear function space. Denote

$$V_0^h = \{\chi \in V^h : \chi(x^k) = 0, \quad \text{for all} x^k \in \partial\Omega_h\}. \tag{5}$$

Then the finite element approximation of (2) is as follows:

$$\min_{v_h \in V_0^h} J(v_h), \tag{6}$$

where

$$J(v_h) = \frac{1}{p} \int_{\Omega_h} |\nabla v_h|^p - \int_{\Omega_h} f v_h. \tag{7}$$

The problem (6) has a unique solution which is an approximative solution of the system (1).

General iterative formula solving the unconstrained optimization such as the problem (6) is as follows

$$u_{n+1} = u_n + \alpha_n d_n, \tag{8}$$

where $u_n \in V_0^h$ is the current approximative solution, and $\alpha_n$ is step length on search direction $d_n$. Search direction $d_n$ is diversified, for example, the steepest descent one (see [16]), hybrid conjugate gradient ones (see [17]) and so on. $\alpha_n$ is usually decided by some line searches, including exact and inexact searches. It is well known that a line search procedure leads to many computations which are dedicated to obtain objective function values and gradients. In order to increase efficiencies of these algorithms, line search procedure in algorithm design should be avoided to the best of one's ability. Based on the thought, for the following unconstrained problem:

$$\min_{x\in R^n} f(x), \tag{9}$$

many algorithms are presented. Dixon [18] proposed a conjugate direction method without line search. Barzilai and Borwein [19] presented a gradient method without line search which is called BB method. Its step length has certain quasi-Newton property. Since the method often requires less computational work and it is efficient on numerical testing, it has now gotten a great deal of attention in optimization community. The attention includes step length designing, convergent analysis, numerical experiments and applications etc. One can refer to [20–27] and the references therein for details. Convergent properties of some conjugate gradient methods without line search were investigated in [18,28,29]. Based on supervisor and searcher cooperation, Liu and Dai [30] presented an algorithm without line search. Numerical effect of the algorithm is good. Vrahatis et al. [27] proposed a class of gradient algorithm with adaptive stepsize for the problem Eq. (9) and Shi [31] proved its convergence. Especially, Shi and Shen [32] proposed a new descent method without line search.

In [16], the problem how to reduce computational time consuming on line search is discussed. The authors proved convergence of a descent algorithm with fixed step length solving the p-Laplacian. But the fixed step length depends on two parameters $M$ and $\sigma$. From the proof of Theorem 2 in [16] we can see that the two parameters are difficult to define in real computing. In fact, no numerical experiment is implemented on algorithm with the fixed step length in [16]. In this paper, in order to increase efficiency of algorithm solving the problem Eq. (6), we will propose some algorithms without line search.

The rest of this paper is organized as follows. In Section 2, some preliminaries are given. In Section 3, some algorithms without line search for the p-Laplacian are proposed. Their numerical experiments are performed in Section 4. For dealing with the p-Laplacian with large $p$, two corresponding algorithms are presented, in which continuation method is used, and

their numerical testing is implemented. In Section 5, some step length figures are given. We analyze why the new algorithms is so efficient from these figures. We end the paper with some conclusions and discussions.

## 2. Preliminaries

Obviously, the functional $J(v)$ given by (3) is strictly convex for $1 < p < +\infty$. Further, the problem (2) has a unique solution.

Now we introduce some definitions (See [16] also).

The steepest descent direction $w$ of $J(v)$ is defined such that

$$J'(v)(w) = -\|J'(v)\|_*, \quad \|w\| = 1, \tag{10}$$

where $\|\cdot\|$ be a norm in $V_0^h$ and $\|\cdot\|_*$ is the standard dual norm given by

$$\|J'(v)\|_* = \sup_{u \in V_0^h} \frac{|J'(v)(u)|}{\|u\|}.$$

For convenience, when computing descent direction $w$, we formulate our algorithm using the $H_0^1(\Omega)$ norm, which is the same as the norm in [16]. The steepest descent direction $w_n$ in $V_0^h$ is obtained by solving the following equation:

$$\int_{\Omega_h} \nabla w_n \nabla v = -\int_{\Omega_h} |\nabla u_n|^{p-2} \nabla u_n \nabla v + \int_{\Omega_h} f v, \quad \forall v \in V_0^h. \tag{11}$$

Based on degenerate characteristic of the Eq. (1), the weighted preconditioned steepest descent direction is proposed. The direction is gotten by solving the following equation:

$$\int_{\Omega_h} (\epsilon + |\nabla u_n|^{p-2}) \nabla w_n \nabla v = -\int_{\Omega_h} |\nabla u_n|^{p-2} \nabla u_n \nabla v + \int_{\Omega_h} f v, \quad \forall v \in V_0^h, \tag{12}$$

where $\epsilon$ is a small positive constant. The direction is the steepest descent one in $V_0^h$ and the space is equipped with a weighted norm $\|\cdot\|_{\epsilon,u,p}^2 = \int_{\Omega_h} (\epsilon + |\nabla u|^{p-2}) |\nabla \cdot|^2$. Obviously, $V_0^h \hookrightarrow H_0^1(\Omega)$.

For the unconstrained optimization problem Eq. (9), Shi and Shen [32] proposed the descent algorithm without line search. The algorithm is only required to estimate step length at each iteration except for the first iteration and line searches are not necessary. Its theoretic properties are very good, but its numerical testing and applications to functional minimization problem are not considered. In this paper, we will be focus on its application to the problem Eq. (6). Exact line search is executed at the first iteration and from the second iteration on, step lengths of the algorithm are obtained by the estimative formula in [32], that is, step lengths are defined by the following rule $(\mathcal{R})$:

when $k = 1, \alpha_k$ satisfies

$$\min_{\alpha \geq 0} f(x_k + \alpha d_k); \tag{13}$$

when $k \geq 2, \alpha_k$ is gotten by

$$\alpha_k = -\frac{g_k^T d_k}{L_k |d_k|^2}, \tag{14}$$

where $x_k$, $\alpha_k$, $d_k$, $g_k$, $L_k$ and $|\cdot|^2$ are the current iterative point, a positive step length, a search direction, gradient of $f(x_k)$, a constant related to Lipschitz constant of $f(x)$ and inner product in $R^n$, respectively.

There are several ways to get parameter $L_k$ (see [32]), for example,

$$L_k = \frac{|g_k - g_{k-1}|}{|x_k - x_{k-1}|}. \tag{15}$$

Convergence of corresponding algorithm is given in [32]. The rule $(\mathcal{R})$ can be used in the steepest descent algorithm solving the p-Laplacian.

## 3. The steepest descent algorithm without line search

Suppose that $u_n$ is current approximative solution of the problem Eq. (6). Next iterative point is updated by Eq. (8). Let $\varphi_i, i = 1, \ldots, \mathcal{N}$ be the basis of the finite element space $V_0^h$ which was defined by Eq. (5), and $w_n = W_n^T \Phi, u_n = U_n^T \Phi$, in which $W_n, U_n$ are real number vectors and $\Phi$ is a function vector, denoted by

$$W_n = \begin{pmatrix} w_n^{(1)} \\ w_n^{(2)} \\ \vdots \\ w_n^{(\mathcal{N})} \end{pmatrix}, \quad \Phi = \begin{pmatrix} \varphi_1 \\ \varphi_2 \\ \vdots \\ \varphi_{\mathcal{N}} \end{pmatrix}, \quad U_n = \begin{pmatrix} u_n^{(1)} \\ u_n^{(2)} \\ \vdots \\ u_n^{(\mathcal{N})} \end{pmatrix}.$$

Now we give the steepest descent algorithm without line search for the p-Laplacian, in which search direction $d_n$ is the weighted preconditioned steepest descent direction, $w_n$ defined by (12) and step length $\alpha_n$ gotten by the rule $(\mathcal{R})$.

**Algorithm 1.** Input: a given initial solution $u_0$ and two small positive constants $\epsilon_1$ and $\epsilon_2$.

Output: iterative number, computing time. If exact solution of testing problem exists, also output finite element error and interpolation error.

*Step 1.* Obtaining the steepest descent direction $w_n$ by solving the Eq. (12), in which $\epsilon$ is replaced by $\epsilon_1$.
*Step 2.* If $\|w_n\|_{\epsilon_1,u_n,p} < \epsilon_2$, stop.
*Step 3.* Computing step length $\alpha_n$ according to the following rule $(\tilde{\mathcal{R}})$:
when $n = 0, \alpha_n$ satisfies

$$\min_{\alpha\geq 0} J(u_n + \alpha w_n); \tag{16}$$

when $n \geq 1, \alpha_n$ is obtained by

$$\alpha_n = L_n^{-1}, \tag{17}$$

where $L_n$ is defined by

$$L_n = \frac{|W_{n-1} - W_n|}{|U_n - U_{n-1}|}. \tag{18}$$

*Step 4.* Updating iterative solution. Let $u_n := u_n + \alpha_n w_n, n := n + 1$; return Step 1.

**Remark 1.** Eq. (12) can be solved by fast multigrid solvers.

In [16], convergence of the weighted preconditioned steepest descent algorithm with exact line search has been proved. In order to control convergence of Algorithm 1, we hope that the functional $J(u_n)$ is sufficiently decreasing at every iteration. Hence, we get the following Algorithm 2, in which step length is gotten by the rule $(\tilde{\mathcal{R}})$ when $J(u_n)$ is decreasing, or it satisfies exact line search Eq. (16).

**Algorithm 2.** Input: a given initial solution $u_0$, two small positive constants $\epsilon_1, \epsilon_2$ and a positive constant $C_0$.

Output: iterative number, computing time. If exact solution of testing problem exists, also output finite element error and interpolation error.

*Step 1* Obtaining the steepest descent direction $w_n$ by solving the Eq. (12), in which $\epsilon$ is replaced by $\epsilon_1$.
*Step 2* If $\|w_n\|_{\epsilon_1,u_n,p} < \epsilon_2$, stop.
*Step 3* Defining step length $\alpha_n$.
If

$$J(u_{n-1}) - J(u_n) \geq C_0 |W_{n-1}|^2, \tag{19}$$

then $\alpha_n$ is defined by the rule $(\tilde{\mathcal{R}})$ in Algorithm 1; or it is gotten by exact line search Eq. (16).
*Step 4* Updating iterative solution. Let $u_n := u_n + \alpha_n w_n$, $n := n + 1$; return Step 1.

**Remark 2.** We can deduce that how to choose an appropriate constant $C_0$ is not an easy thing from the proof of convergence in [16].

**Remark 3.** From the numerical experiments in [16] we know that $\epsilon_1 = 10^{-4}$ in Algorithm 2 is a good choice. $\epsilon_2$ in Algorithm 2 is a computing precision which can be decided freely. In all numerical experiments in this paper, $\epsilon_2$ is $10^{-4}$.

From our numerical experiments we found out that Algorithms 1 and 2 can only solve the p-Laplacian for a narrow range of $p$. For Example 1 in Section 4, they are efficient only when $p \in [1.5, 7]$. For $p$ being the narrow range, Algorithms 1 and 2 may convergent slowly or diverge. Convergence of the two algorithms is related to the parameter $L_n$ in Eq. (15). When $p$ is outside the narrow range, we have to search for a new formula for computing $L_n$. But how to define the parameter $L_n$ is not an easy thing. So for solving the p-Laplacian in which $p$ belongs to a wide range, we attempt to use continuation method in Algorithms 1 and 2. Accordingly, we get the following Algorithms 3 and 4.

**Algorithm 3.** Input: a given initial solution $u_0$, three small positive constants $\epsilon_1, \epsilon_2, \epsilon_3 (\epsilon_2 < \epsilon_3)$, an initial constant $p_1 (p \geq p_1 > 1)$ and a positive constant $m$ which satisfies $mod(p - p_1, m) = 0$.

Output: iterative number, computing time. If exact solution of testing problem exists, also output finite element error and interpolation error.

*Step 1* Obtaining the steepest descent direction $w_n$ by solving the equation

$$\int_{\Omega_h}(\epsilon_1+|\nabla u_n|^{p_1-2})\nabla w_n\nabla v=-\int_{\Omega_h}|\nabla u_n|^{p_1-2}\nabla u_n\nabla v+\int_{\Omega_h}fv,\quad\forall\ v\in V_0^h. \tag{20}$$

*Step 2* If $p_1=p$, then stop when $\|w_n\|_{\epsilon_1,u_n,p_1}<\epsilon_2$, or go to Step 3.
If $p_1\neq p$, then compute $\|w_n\|_{\epsilon_1,u_n,p_1}$. When $\|w_n\|_{\epsilon_1,u_n,p_1}<\epsilon_3$, let $k:=k+1, p_1:=p_1+m\cdot k$, and return to Step 1, or go to Step 3.

*Step 3* Computing step length $\alpha_n$ according to the rule $(\tilde{\mathcal{R}})$ in Algorithm 1.

*Step 4* Updating iterative solution. Let $u_n:=u_n+\alpha_n w_n$, $n:=n+1$; return Step 1.

We can formulate Algorithm 4 only by replacing Step 3 in Algorithm 3 with Step 3 in Algorithm 2.

From the following numerical results we can see that continuation method in Algorithms 3 and 4 can help Algorithms 1 and 2 to find a good initial solution that is very important to ensure that iterative course is steady, convergent and fast.

## 4. Numerical experiments

We test Algorithms 1–3 and 4, in which piecewise linear triangle finite element approximation is used. The program language is Fortran 90. Zero is always used as an initial solution except Tables 3 and 5. A random value in $(0,1)$ is used as initial solution in Tables 3 and 5. The descent direction $w_n$ is computed by an algebraic multigrid solver (AMG). The stopping rule for the AMG solver is to reduce the relative defect to $10^{-8}$ and the maximin V-Cycles is 50. A 0.618-section algorithm is used as an exact line search procedure. The parameters $\epsilon_1$ and $\epsilon_2$ are all equal to $10^{-4}$. We set the parameters as $C_0=10^{-3}$ in Algorithms 2 and 4, $\epsilon_3=10^{-2}$ in Algorithms 3 and 4, respectively. Simultaneously, discretion accuracy of object functional and solution can be obtained. The stopping condition is the same as one in [16]. All computations in this paper are carried out in an SGI Indigo workstation. Results of three numerical examples are listed in the tables below, in which *C1*,*C2*, *C3* and *C4* represent the meshes with 1601,6221,24444 and 97118 nodes, respectively; *WPSD*, *WPCG*, *AG1*, *AG2*, *AG3* and *AG4* denote the weighted preconditioned steepest descent algorithm with exact line search [19], the hybrid conjugate gradient algorithm with exact line search [17], Algorithms 1, 2, 3 and 4 in this paper, respectively; *ItN*, *CPU*, $\|\cdot\|$, $u$, $u_h, u_I$ and $*-n$ denote iterative numbers, computing time, $L^2$-norm, exact solution, finite element one, interpolation one and $*\times 10^{-n}$, respectively.

**Example 1.** $\Omega=\{(x,y)|r^2=x^2+y^2<1\}, f=1$. The exact solution is

$$u=u(r)=\frac{p-1}{p}\left(\frac{1}{2}\right)^{\frac{1}{p-1}}(1-r^{\frac{p}{p-1}}). \tag{21}$$

Its numerical results are listed in Tables 1–5.

**Example 2.** $\Omega=\{(x,y)|x^2+y^2<1\},\ f=e^{xy}$. Since analytic solution of the problem is unknown, we only list iterative numbers and CPU time in Tables 6 and 7, so is Example 3.

**Example 3.** $\Omega=\{(x,y)|x^2+y^2<1\},\ f=2(x+y-x^2-y^2)$.

Its numerical results are reported in Tables 8 and 9.

It is obvious that convergence of the four new algorithms are almost independent of meshes, and Algorithms 3 and 4 can solve the p-Laplacian with large $p$ and their convergent rate is almost linear. These good properties are summarized by Huang, Li and Liu [16]. Generally, although the new algorithms need more iteration numbers than WPSD and WPCG, most iterations are lower-cost. Therefore, from these tables, we can see that CPU time of Algorithms 1 and 3 is less than that of WPSD and WPCG. The fact shows that a good algorithm without line search can cut down some CPU time. We can also see that since in Algorithms 1 and 3, line search are used in their iterative course except the first iteration, CPU time of Algorithms 2 and 4 is more than that of Algorithms 1 and 3, respectively. Of course, CPU time of Algorithms 2 and 4 is usually less than that of WPSD and WPCG. In addition, according to Tables 3 and 5, we can see that the four new algorithms are also very effective for a random initial solution.

## 5. Analysis on step lengths

For visually observing the fast convergent cause of these new algorithms, we plot step length figures gotten by using WPSD and the new algorithms to solve Example 1. We only display figures under C1 mesh because figures gotten by the same algorithm have similar characteristics for the four meshes.

**Table 1**
Numerical comparisons on Example 1 when $p = 1.3$.

| | C1 | | | C2 | | |
|---|---|---|---|---|---|---|
| | WPSD | AG1 | AG2 | WPSD | AG1 | AG2 |
| ItN | 10 | 18 | 12 | 10 | 14 | 11 |
| CPU | 0m10s | 0m03s | 0m05s | 0m38s | 0m08s | 0m19 |
| $\|u - u_h\|$ | 9.76-5 | 9.56-5 | 9.33-5 | 2.51-5 | 2.68-5 | 3.01-5 |
| $\|u_h - u_I\|$ | 3.98-5 | 3.88-5 | 3.57-5 | 9.52-6 | 1.31-5 | 1.70-5 |
| | C3 | | | C4 | | |
| | WPSD | AG1 | AG2 | WPSD | AG1 | AG2 |
| ItN | 10 | 15 | 12 | 10 | 22 | 12 |
| CPU | 2m34s | 0m37s | 0m59s | 10m25s | 3m44s | 5m30s |
| $\|u - u_h\|$ | 7.38-6 | 6.74-6 | 6.72-6 | 3.49-6 | 1.76-6 | 3.28-6 |
| $\|u_h - u_I\|$ | 3.43-6 | 3.17-6 | 3.14-6 | 2.76-6 | 8.87-7 | 3.59-6 |

**Table 2**
Numerical comparisons on Example 1 when $p = 5$.

| | C1 | | | C2 | | |
|---|---|---|---|---|---|---|
| | WPSD | AG1 | AG2 | WPSD | AG1 | AG2 |
| ItN | 9 | 11 | 10 | 9 | 11 | 10 |
| CPU | 0m09s | 0m02s | 0m03s | 0m34s | 0m06s | 0m09s |
| $\|u - u_h\|$ | 5.99-4 | 5.96-4 | 5.99-4 | 1.52-4 | 1.50-4 | 1.52-4 |
| $\|u_h - u_I\|$ | 7.60-5 | 7.09-5 | 7.41-5 | 1.86-5 | 1.67-5 | 1.83-5 |
| | C3 | | | C4 | | |
| | WPSD | AG1 | AG2 | WPSD | AG1 | AG2 |
| ItN | 8 | 11 | 10 | 8 | 11 | 10 |
| CPU | 1m59s | 0m26s | 0m41s | 7m59s | 1m51s | 2m48s |
| $\|u - u_h\|$ | 3.56-5 | 3.75-5 | 3.83-5 | 9.24-6 | 1.00-5 | 1.17-5 |
| $\|u_h - u_I\|$ | 5.72-6 | 5.15-6 | 7.54-6 | 1.33-6 | 2.59-6 | 6.61-6 |

**Table 3**
Numerical comparisons on Example 1 when $p = 5$ and initial solution is random.

| | C1 | | | C2 | | |
|---|---|---|---|---|---|---|
| | WPSD | AG1 | AG2 | WPSD | AG1 | AG2 |
| ItN | 17 | 22 | 17 | 18 | 30 | 31 |
| CPU | 0m16s | 0m03s | 0m03s | 1m07s | 0m18s | 0m36s |
| $\|u - u_h\|$ | 5.94-4 | 5.96-4 | 5.96-4 | 1.54-4 | 1.51-4 | 1.51-4 |
| $\|u_h - u_I\|$ | 7.77-5 | 7.68-5 | 7.67-5 | 1.85-5 | 1.91-5 | 1.93-5 |
| | C3 | | | C4 | | |
| | WPSD | AG1 | AG2 | WPSD | AG1 | AG2 |
| ItN | 18 | 20 | 18 | 20 | 33 | 20 |
| CPU | 4m28s | 0m44s | 1m07s | 19m53s | 5m13s | 4m57s |
| $\|u - u_h\|$ | 3.71-5 | 3.72-5 | 3.71-5 | 9.41-6 | 9.50-6 | 9.70-6 |
| $\|u_h - u_I\|$ | 4.76-6 | 4.74-6 | 4.84-6 | 1.36-6 | 1.88-7 | 1.46-6 |

With $p = 5$, we obtained step lengths under C1 mesh by carrying out the three algorithms WPSD, AG1 and AG2, respectively. These step lengths are showed in Fig. 1. Relevant iterative numbers and CPU time etc. can be found in Table 2. From Fig. 1, we can see that step lengths gotten by the three algorithms all exist in interval $[10^{-4}, 0.7]$. Iteration numbers of AG1 and AG2 are two and one more than that of WPSD, respectively. Step lengths of AG1 after 5 iterations and AG2 after 3 iterations exist in interval $[10^{-2}, 0.7]$, so are step lengths of WPSD after 2 iterations. Therefore, exact step lengths in AG1 and AG2 can be substituted properly by step lengths gotten by formula (14).

With $p = 150$, we obtained step lengths under C1 mesh by carrying out the three algorithms WPSD, AG3 and AG4, respectively. These step lengths are showed in Fig. 2. Relevant iterative numbers and CPU time etc. can be found in Table 4. From Fig. 2, we can see that step lengths of WPSD after the 23rd iteration oscillate near $10^{-2}$. Step lengths characteristic of AG4 is similar to that of AG3. Apart from the step lengths of the first iteration for every new $p_1$, their step lengths are almost in

**Table 4**
Numerical comparisons on Example 1 when $p = 150, p_1 = 5$ and $m = 5$.

| | C1 | | | C2 | | |
|---|---|---|---|---|---|---|
| | WPSD | AG3 | AG4 | WPSD | AG3 | AG4 |
| ItN | 133 | 135 | 136 | 98 | 140 | 134 |
| CPU | 2m13s | 0m20s | 0m22s | 6m34s | 1m29s | 1m33s |
| $\|u - u_h\|$ | 4.13-3 | 4.13-3 | 4.13-3 | 1.38-3 | 1.38-3 | 1.38-3 |
| $\|u_h - u_l\|$ | 3.33-3 | 3.33-3 | 3.33-3 | 1.18-3 | 1.18-3 | 1.18-3 |
| | C3 | | | C4 | | |
| | WPSD | AG3 | AG4 | WPSD | AG3 | AG4 |
| ItN | 88 | 129 | 129 | 105 | 123 | 121 |
| CPU | 23m58s | 6m00s | 6m31s | 115m48s | 24m54s | 26m28s |
| $\|u - u_h\|$ | 4.23-4 | 4.23-4 | 4.23-4 | 1.25-4 | 1.26-4 | 1.26-4 |
| $\|u_h - u_l\|$ | 3.75-4 | 3.75-4 | 3.75-4 | 1.13-4 | 1.13-4 | 1.13-4 |

**Table 5**
Numerical comparisons on Example 1 when $p = 4005, p_1 = 5$, $m = 20$ and initial solution is random.

| | C1 | | C2 | |
|---|---|---|---|---|
| | AG3 | AG4 | AG3 | AG4 |
| ItN | 421 | 411 | 332 | 582 |
| CPU | 1m01s | 2m18s | 6m32s | 20m09s |
| $\|u - u_h\|$ | 6.63-3 | 6.63-3 | 3.16-3 | 3.16-3 |
| $\|u_h - u_l\|$ | 5.87-3 | 5.87-3 | 2.97-3 | 2.97-3 |

**Table 6**
Numerical comparisons on Example 2 when $p = 5$.

| | C1 | | | C2 | | |
|---|---|---|---|---|---|---|
| | WPSD | AG1 | AG2 | WPSD | AG1 | AG2 |
| ItN | 13 | 17 | 15 | 13 | 17 | 15 |
| CPU | 0m13s | 0m03s | 0m03s | 0m49s | 0m10s | 0m12s |
| | C3 | | | C4 | | |
| | WPSD | AG1 | AG2 | WPSD | AG1 | AG2 |
| ItN | 13 | 17 | 15 | 13 | 17 | 15 |
| CPU | 3m22s | 0m42s | 0m54s | 13m36s | 3m02s | 3m48s |

**Table 7**
Numerical comparisons on Example 2 when $p = 40, p_1 = 5$ and $m = 5$.

| | C1 | | | C2 | | |
|---|---|---|---|---|---|---|
| | WPCG | AG3 | AG4 | WPCG | AG3 | AG4 |
| ItN | 29 | 59 | 57 | 31 | 59 | 57 |
| CPU | 0m30s | 0m08s | 0m11s | 2m05s | 0m38s | 0m41s |
| | C3 | | | C4 | | |
| | WPCG | AG3 | AG4 | WPCG | AG3 | AG4 |
| ItN | 33 | 59 | 57 | 39 | 59 | 57 |
| CPU | 9m08s | 2m44s | 2m56s | 44m13s | 11m52s | 12m39s |

interval $[10^{-2}, 0.1]$. Step length of the first iteration for every new $p_1$ in AG3 and AG4 is in interval $[10^{-5}, 10^{-2}]$. These small step lengths lead to oscillation of their step length figures. When $p_1 = p$ in AG3 and AG4, variety of step lengths of AG4 is almost the same as that of AG3. This fact can be easily detected from trail of the second and third small figures in Fig. 2. In the whole, most step lengths of AG3 and AG4 exist in the same interval as those of WPSD, which is $[10^{-2}, 0.1]$.

To sum up, the essential reason that these new algorithms can converge quickly is that under the steepest descent direction, step length obtained by the formula (14) can approximate the one gotten by the exact line search.

**Table 8**
Numerical comparisons on Example 3 when $p = 5$.

| | C1 | | | C2 | | |
|---|---|---|---|---|---|---|
| | WPCG | AG1 | AG2 | WPCG | AG1 | AG2 |
| ItN | 16 | 19 | 19 | 17 | 18 | 22 |
| CPU | 0m15s | 0m02s | 0m05s | 1m05s | 0m10s | 0m21s |
| | C3 | | | C4 | | |
| | WPCG | AG1 | AG2 | WPCG | AG1 | AG2 |
| ItN | 19 | 20 | 26 | 24 | 21 | 38 |
| CPU | 4m52s | 0m45s | 1m44s | 24m40s | 3m19s | 10m39s |

**Table 9**
Numerical comparisons on Example 3 when $p = 33, p_1 = 3$ and $m = 3$.

| | C1 | | | C2 | | |
|---|---|---|---|---|---|---|
| | WPCG | AG3 | AG4 | WPCG | AG3 | AG4 |
| ItN | 58 | 73 | 75 | 63 | 73 | 74 |
| CPU | 0m59s | 0m12s | 0m15s | 4m18s | 0m49s | 1m04s |
| | C3 | | | C4 | | |
| | WPCG | AG3 | AG4 | WPCG | AG3 | AG4 |
| ItN | 74 | 75 | 74 | 75 | 91 | 91 |
| CPU | 20m21s | 3m34s | 4m30s | 80m16s | 17m27s | 25m39s |

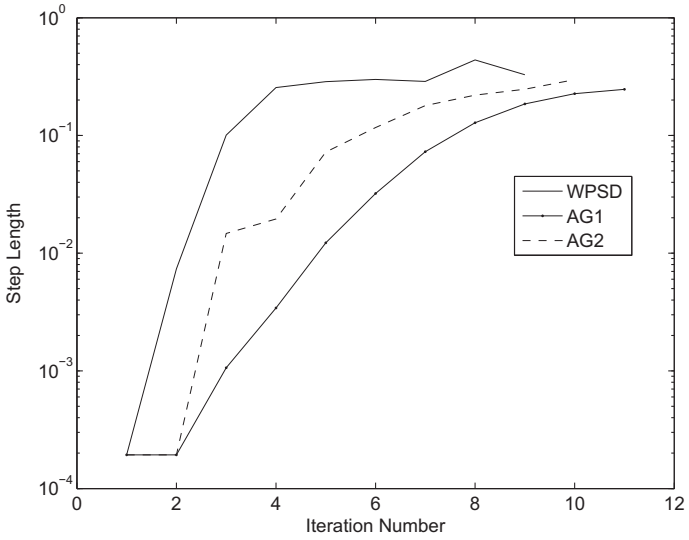



**Fig. 1.** This is step length figure of WPSD,AG1 and AG2.

## 6. Conclusions and discussions

Based on the steepest descent direction [16] and iterative algorithm without line search [32], we propose the weighted preconditioned steepest descent algorithm without line search for the p-Laplacian. In order to control convergence of the algorithm, its improved version is gotten, in which sufficiently decreasing condition of the objection functional is required at every iteration step. The two algorithms need less computational time than WPSD [16] and WPCG [17]. Generally, the two algorithms can only solve the p-Laplacian with $p$ belonging to a narrow range. Their modified versions can deal with the equation with very large $p$ if continuation method is applied in them. So we get additional two algorithms. Lots of numerical experiments are implemented on the two algorithms with continuation technique. Numerical results show that computational time of the two algorithms is also less than those of WPSD and WPCG when $p$ is large. Therefore, by comparing

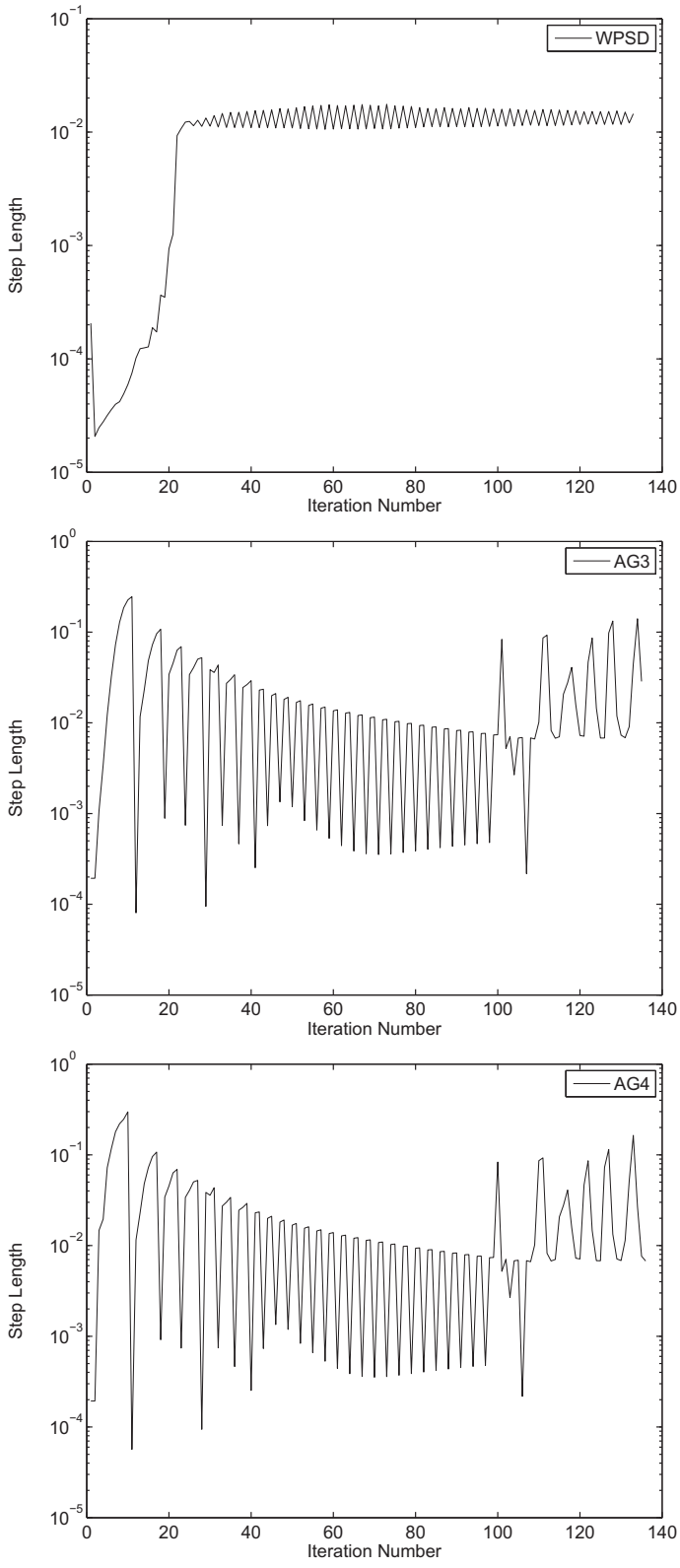



**Fig. 2.** These are step length figures of WPSD, AG3 and AG4 from above to below.

numerical results of the four new algorithms with that of WPSD and WPCG, we can easily conclude that these algorithms without line search can cut down the computational time solving the p-Laplacian when $p$ is not close $1^+$. At last, for observing the reason of fast convergence of these new algorithms, we plot step length figures of WPSD and these new algorithms for solving Example 1. Through these figures, we conclude that if the search direction is the steepest descent one, step lengths obtained by a good approximative formula can substitute exact step lengths properly.

## Acknowledgements

The authors are indebted to Prof. Yun-qing Huang for his guidance and Prof. Ruo Li for providing the code of the algorithm in the reference [16]. They also thank the anonymous referees and editors, whose comments and suggestions improved this paper greatly. This work is supported by Foundation of Hunan Science and Technology Committee (2010FJ6020) and Doctor Foundation of Xiangtan University (D2QDZ10).

## References

[1] J.R. Philip, N-diffusion, Aust. J. Phys. 14 (1961) 1–13.
[2] C. Atkinson, C.R. Champion, Some boundary value problems for the equation $\nabla \cdot (|\nabla\phi|^N \nabla\phi) = 0$, Q. J. Mech. Appl. Math. 37 (1984) 401–419.
[3] C. Atkinson, C.W. Jones, Similarity solutions in some nonlinear diffusion problems and in boundary-layer flow of a pseudo plastic fluid, Q. J. Mech. Appl. Math. 27 (1974) 193–211.
[4] M. Ainsworth, D. Kay, The approximation theory for the p-version finite element method and application to non-linear elliptic PDEs, Numer. Math. 83 (1999) 351–388.
[5] J. Baranger, H.E. Amri, Estimateurs a posteriori d'erreur pour le calcul adaptatif d'ecoulements quasi-Newtoniens, R.A.I.R.O. M2AN 25 (1991) 31–48.
[6] J.W. Barrett, W.B. Liu, Finite element approximation of the p-Laplacian, Math. Comput. 61 (1993) 523–537.
[7] J.W. Barrett, W.B. Liu, Finite element approximation of some degenerate quasi-linear problems, Lect. Notes Math. 303 (1994) 1–16.
[8] J.W. Barrett, W.B. Liu, Quasi-norm error bounds for finite element approximation of quasi-Newtonian flows, Numer. Math. 68 (1994) 437–456.
[9] R. Bermejo, J. Infante, A multigrid algorithm for the p-Laplacian,, SIAM J. Sci. Comput. 21 (2000) 1774–1789.
[10] S.S. Chow, Finite element error estimates for non-linear elliptic equations of monitone type, Numer. Math. 54 (1988) 373–393.
[11] X.C. Tai, J.C. Xu, Global convergence of subspace correction methods for convex optimization problems, Math. Comput. 74 (2002) 105–124.
[12] W.B. Liu, N. Yan, Quasi-norm local error estimates for finite element approximation of p-Lapalacian, SIAM J. Numer. Anal. 39 (2001) 100–127.
[13] W.B. Liu, N. Yan, Quasi-norm a posteriori error estimates for non-conforming finite element approximation of p-Laplacian, Numer. Math. 89 (2001) 341–378.
[14] W.B. Liu, N. Yan, On quasi-norm interpolation error estimates and a posteriori error estimates for p-Laplacian, SIAM J. Numer. Anal. 40 (2003) 1870–1895.
[15] P.G. Ciarlet, The Finite Method for Elliptic Problems, North-Holland, Amsterdam, 1978.
[16] Y.Q. Huang, R. Li, W.B. Liu, Preconditioned descent algorithms for p-Laplacian, J. Sci. Comput. 32 (2007) 343–371.
[17] G.M. Zhou, Y.Q. Huang, C.S. Feng, Preconditioned hybrid conjugate gradient algorithm for p-Laplacian, Int. J. Numer. Anal. Model. 2 (2005) 123–130 (Special Issue).
[18] L.C.W. Dixon, Conjugate directions without line searches, J. Inst. Math. Appl. 11 (1973) 317–328.
[19] J. Barzilai, J.M. Borwein, Two point step size gradient methods, IMA J. Numer. Anal. 8 (1988) 141–148.
[20] E.G. Birgin, I. Chambouleyron, J.M. Martinez, Estimation of the optical constants and the thickness of thin films using unconstrained optimization, J. Comput. Phys. 151 (1999) 862–880.
[21] Y.H. Dai, Y. Yuan, Alternate minimization gradient method, IMA J. Numer. Anal. 23 (2003) 377–393.
[22] Y.H. Dai, Y.J. Yuan, Y. Yuan, Modified two-point stepsize gradient methods for unconstrained optimization, Comput. Optim. Appl. 22 (2002) 103–109.
[23] Y.H. Dai, H. Zhang, An adaptive two-point stepsize gradient algorithm, Numer. Algorithms 27 (2001) 377–385.
[24] R. Fletcher, Low storage methodsfor unconstrained optimization, Lect. Appl. Math. (AMS) 25 (1990) 165–179.
[25] M. Raydan, The Barzilai and Borwein method for the large scale unconstrained minimization problem, SIAM J. Optim. 7 (1997) 26–33.
[26] M. Raydan, On the Barzilai and Borwein gradient choice of steplength for the gradient method, IMA J. Numer. Anal. 13 (1993) 321–326.
[27] M.N. Varhatis, G.S. Amdroulakis, J.N. Lambrinos, A class of gradient unconstrained minimization algorithms with adaptive stepsize, J. Comput. Appl. Math. 114 (2000) 367–386.
[28] X.D. Chen, J. Sun, Global convergence of a two-parameter family of conjugate gradient methods without line search, J. Comput. Appl. Math. 146 (2002) 37–45.
[29] J. Sun, J.P. Zhang, Global convergence of conjugate gradient methods without line search, Ann. Oper. Res. 103 (2001) 161–173.
[30] W.B. Liu, Y.H. Dai, Minimization algorithms based on supervisor and searcher cooperation, JOTA 2 (2001) 359–379.
[31] Z.J. Shi, Convergence of line search methods for unconstrained optimization, Appl. Math. Comput. 157 (2004) 393–405.
[32] Z.J. Shi, J. Shen, Convergence of descent method without line search, Appl. Math. Comput. 167 (2005) 94–107.